

Competitive Programming

Section 16 Editorial: DFS, Components, And Cycles

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	4
2	DFS reminder	5
3	A. Subtree Sizes	6
3.1	Restatement	6
3.2	Constraints and traps	6
3.3	Observations	6
3.4	Walk-through	6
3.5	Algorithm	7
3.6	Correctness proof	7
3.7	Complexity	8
3.8	Full C++ solution	8
3.9	Full Python solution	9
3.10	Common wrong answers	10
3.11	Tests that matter	10
4	B. Undirected Cycle	11
4.1	Restatement	11
4.2	Constraints and traps	11
4.3	Observations	11
4.4	Walk-through	11
4.5	Algorithm	12
4.6	Correctness proof	12
4.7	Complexity	13
4.8	Full C++ solution	13
4.9	Full Python solution	14
4.10	Common wrong answers	14
4.11	Tests that matter	15
5	C. Directed Cycle	16
5.1	Restatement	16
5.2	Constraints and traps	16
5.3	DFS color reminder	16
5.4	Walk-through	16
5.5	Algorithm	17
5.6	Correctness proof	17
5.7	Complexity	18
5.8	Full C++ solution	18
5.9	Full Python solution	19
5.10	Common wrong answers	20
5.11	Tests that matter	20
6	D. Component Audit	21
6.1	Restatement	21

6.2	Constraints and traps	21
6.3	Observation and walk-through	21
6.4	Algorithm	21
6.5	Correctness proof	22
6.6	Complexity	23
6.7	C++ implementation notes	23
6.8	Python implementation notes	23
6.9	Full C++ solution	23
6.10	Full Python solution	24
6.11	Common wrong answers	25
6.12	Tests that matter	25
7	Stress testing	26
8	What to carry forward	27

1 Editorial overview

The four local tasks use DFS in different modes:

Problem	DFS mode
A. Subtree Sizes	root a tree and process postorder
B. Undirected Cycle	traversal with parent skipping
C. Directed Cycle	traversal with active/finished colors
D. Component Audit	collect component statistics while traversing

The common theme is state:

What do we know when entering a vertex?
What do we know after all children are finished?
What does an edge to an already seen vertex mean?

2 DFS reminder

DFS can be written recursively or iteratively. The reference solutions are iterative to avoid recursion-depth failures on long chains.

Basic iterative traversal:

```
mark start
push start
while stack not empty:
    pop u
    for each neighbor v:
        if v is unvisited:
            mark v
            push v
```

Postorder work requires processing descendants before ancestors. Directed cycle detection requires knowing whether a vertex is currently active in the DFS stack. Those two needs are why this section uses more than the simplest stack template.

3 A. Subtree Sizes

3.1 Restatement

Root the given tree at vertex 1. For every vertex v , compute how many vertices are in the subtree of v , including v itself.

3.2 Constraints and traps

n can be large, so the solution must be linear.

Traps:

- the input edges are undirected even though the rooted tree has parent-child direction after rooting;
- the root's parent needs a sentinel value;
- subtree sizes must be accumulated after children are known;
- recursive DFS may overflow on a chain.

3.3 Observations

Once the tree is rooted, every non-root vertex has exactly one parent. The subtree recurrence is:

```
subtree[u] = 1 + sum(subtree[v] for children v of u)
```

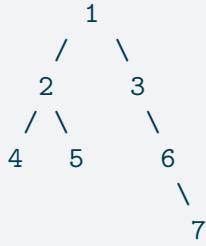
This is a postorder computation. We need children before parent.

The iterative trick:

1. Run a stack traversal from the root, recording **parent** and **order**.
2. Process **order** in reverse.
3. Add each vertex's subtree size to its parent.

3.4 Walk-through

Tree:



If a DFS order is:

1, 3, 6, 7, 2, 5, 4

then reverse order is:

4, 5, 2, 7, 6, 3, 1

Every child appears before its parent in this reversed list, so adding child sizes upward is valid.

3.5 Algorithm

```

build undirected adjacency list
parent[root] = root
stack = [root]
order = []

while stack not empty:
    u = pop
    append u to order
    for v in adj[u]:
        if v == parent[u]:
            continue
        parent[v] = u
        push v

subtree[u] = 1 for all u
for u in reversed(order):
    if u is not root:
        subtree[parent[u]] += subtree[u]

print subtree[1..n]

```

3.6 Correctness proof

Lemma 1. The traversal assigns the correct parent to every non-root vertex.

Proof. In a tree there is exactly one simple path from the root to any vertex. When traversal first reaches a vertex v from u , the edge $u-v$ lies on that root-to- v path, so u is the rooted parent

of v . Because a tree has no cycles, no vertex can be reached later through a different parent. $\square$

Lemma 2. In `reversed(order)`, every vertex appears before its parent.

Proof. A vertex is appended to `order` when it is popped from the DFS stack. Its children are pushed only after the parent has been popped and appended. Therefore every child is appended after its parent in `order`, and before its parent in `reversed(order)`. $\square$

Lemma 3. When processing a vertex u in `reversed(order)`, `subtree[u]` equals the true subtree size of u .

Proof. By Lemma 2, all children of u were processed earlier in the reversed order. Each child has already added its complete subtree size into u . Starting from 1 for u itself, the value is exactly 1 plus all child subtree sizes. $\square$

Theorem. The algorithm prints the correct subtree size for every vertex.

Proof. Lemma 3 applies to every vertex when it is processed. The algorithm prints the resulting values. $\square$

3.7 Complexity

```
Time:  O(n)
Memory: O(n)
```

3.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
    vector<vector<int>> adj(n);
    for (int i = 0; i < n - 1; i++) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    vector<int> parent(n, -1), order;
    vector<int> st = {0};
```

```

parent[0] = 0;
while (!st.empty()) {
    int u = st.back();
    st.pop_back();
    order.push_back(u);
    for (int v : adj[u]) {
        if (v == parent[u]) continue;
        parent[v] = u;
        st.push_back(v);
    }
}

vector<int> subtree(n, 1);
for (int i = (int)order.size() - 1; i >= 0; i--) {
    int u = order[i];
    if (u != 0) subtree[parent[u]] += subtree[u];
}

for (int i = 0; i < n; i++) {
    if (i) cout << ' ';
    cout << subtree[i];
}
cout << '\n';
return 0;
}

```

3.9 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    adj = [[] for _ in range(n)]
    for _ in range(n - 1):
        u = next(it) - 1
        v = next(it) - 1
        adj[u].append(v)
        adj[v].append(u)

    parent = [-1] * n
    order = []
    stack = [0]
    parent[0] = 0

```

```

while stack:
    u = stack.pop()
    order.append(u)
    for v in adj[u]:
        if v == parent[u]:
            continue
        parent[v] = u
        stack.append(v)

size = [1] * n
for u in reversed(order):
    if u != 0:
        size[parent[u]] += size[u]

print(" ".join(map(str, size)))

if __name__ == "__main__":
    main()

```

3.10 Common wrong answers

- Computing `subtree[u]` before child sizes are ready.
- Forgetting that input edges are undirected.
- Treating vertex 1 as index 1 instead of index 0 after conversion.
- Using recursive DFS without considering a chain of length 200000.

3.11 Tests that matter

Use:

- one vertex;
- a chain;
- a star centered at the root;
- a balanced tree;
- a tree where high-numbered vertices are ancestors of lower-numbered vertices.

4 B. Undirected Cycle

4.1 Restatement

Given a simple undirected graph, print **CYCLIC** if any connected component contains a cycle. Otherwise print **ACYCLIC**.

4.2 Constraints and traps

The graph can be disconnected.

Traps:

- the edge back to the DFS parent is not a cycle;
- all components must be checked;
- this problem says the graph is simple, so parallel edges do not need special handling;
- a component with k vertices and $k-1$ edges can still be a tree only if it is connected, which is why traversal is natural.

4.3 Observations

In an undirected DFS tree, every scanned edge $u-v$ falls into one of these cases:

```
v is parent[u]    -> the edge we used to enter u; ignore
v is unseen       -> tree edge; continue DFS
v is already seen  -> another route to an existing vertex; cycle
```

The third case means there is a path in the DFS tree from v to u , plus the edge $u-v$, forming a cycle.

4.4 Walk-through

Graph:

```
1 -- 2 -- 3
|       |
4 -----
```

DFS may visit:

```
1 -> 2 -> 3 -> 4
```

At 4, neighbor 1 is already seen and is not the parent of 4. The path 1-2-3-4 plus edge 4-1 is a cycle.

4.5 Algorithm

```
build undirected adjacency list
seen = false for all vertices

for every start:
    if seen[start]: continue
    mark start
    stack = [(start, no_parent)]
    while stack not empty:
        u, parent = pop
        for v in adj[u]:
            if v == parent:
                continue
            if seen[v]:
                print CYCLIC and stop
            mark v
            push (v, u)

print ACYCLIC
```

4.6 Correctness proof

Lemma 1. If the algorithm prints **CYCLIC**, the graph contains a cycle.

Proof. The algorithm prints **CYCLIC** only when it sees an edge $u-v$ where v is already seen and v is not the parent of u . Since v was seen in the same DFS component, the DFS tree contains a path between v and u . This path does not use the edge $u-v$ as the parent edge. Adding edge $u-v$ forms a cycle. $\square$

Lemma 2. If an undirected component contains a cycle, the algorithm will print **CYCLIC**.

Proof. Consider any cycle in the component. During DFS, let x be the first vertex of that cycle discovered, and follow DFS tree discoveries until the traversal reaches other cycle vertices. Because the cycle has a closing edge between two already connected vertices that is not simply a child-parent return edge, eventually DFS scans an edge to an already seen non-parent vertex. Then the algorithm prints **CYCLIC**. $\square$

Theorem. The algorithm prints **CYCLIC** exactly when the graph contains an undirected cycle.

Proof. Lemma 1 proves no false positives. Lemma 2 proves no false negatives. The outer loop checks every component. $\square$

4.7 Complexity

Time: $O(n + m)$
Memory: $O(n + m)$

4.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<vector<int>> adj(n);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }

    vector<char> seen(n, false);
    for (int start = 0; start < n; start++) {
        if (seen[start]) continue;
        seen[start] = true;
        vector<pair<int, int>> st = {{start, -1}};
        while (!st.empty()) {
            auto [u, parent] = st.back();
            st.pop_back();
            for (int v : adj[u]) {
                if (v == parent) continue;
                if (seen[v]) {
                    cout << "CYCLIC\n";
                    return 0;
                }
                seen[v] = true;
                st.push_back({v, u});
            }
        }
    }
    cout << "ACYCLIC\n";
    return 0;
}
```

4.9 Full Python solution

```
import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    adj = [[] for _ in range(n)]
    for _ in range(m):
        u = next(it) - 1
        v = next(it) - 1
        adj[u].append(v)
        adj[v].append(u)

    seen = [False] * n
    for start in range(n):
        if seen[start]:
            continue
        seen[start] = True
        stack = [(start, -1)]
        while stack:
            u, parent = stack.pop()
            for v in adj[u]:
                if v == parent:
                    continue
                if seen[v]:
                    print("CYCLIC")
                    return
                seen[v] = True
                stack.append((v, u))
        print("ACYCLIC")

if __name__ == "__main__":
    main()
```

4.10 Common wrong answers

- Reporting a cycle when seeing the parent edge.
- Starting DFS only from vertex 1.
- Using directed-cycle colors and forgetting that undirected edges appear both ways.
- Assuming $m \geq n$ is enough globally; disconnected graphs require component reasoning.

4.11 Tests that matter

Use:

- empty graph;
- forest with many components;
- one triangle;
- cycle in a component not containing vertex 1;
- connected tree with $n-1$ edges.

5 C. Directed Cycle

5.1 Restatement

Given a directed graph, print **CYCLIC** if it contains a directed cycle. Otherwise print **ACYCLIC**.

5.2 Constraints and traps

The graph can be disconnected.

Traps:

- directed cycle detection is not the same as undirected cycle detection;
- an edge to a finished vertex does not prove a cycle;
- an edge to an active vertex does prove a cycle;
- repeated directed edges do not change the yes/no answer.

5.3 DFS color reminder

Use:

```
0 = unvisited
1 = active in current DFS stack
2 = finished
```

When scanning edge $u \rightarrow v$:

```
color[v] == 0: enter v
color[v] == 1: directed cycle found
color[v] == 2: ignore
```

5.4 Walk-through

Graph:

```
1 -> 2 -> 3 -> 4
    ^         |
    +-----+
```

During DFS:

```
active path: 1,2,3,4
edge 4 -> 2 goes to an active vertex
```

This gives the directed cycle:

```
2 -> 3 -> 4 -> 2
```

Now compare:

```
1 -> 2 -> 4
1 -> 3 -> 4
```

When DFS later sees an edge to 4 after 4 is finished, that does not create a cycle. It is just another path into already processed work.

5.5 Algorithm

```
build directed adjacency list
color = 0 for every vertex

for every start:
    if color[start] != 0:
        continue
    color[start] = 1
    stack = [(start, 0)]
    while stack not empty:
        u, i = top frame
        if i == len(adj[u]):
            color[u] = 2
            pop frame
            continue

        v = adj[u][i]
        increment top frame index
        if color[v] == 0:
            color[v] = 1
            push (v, 0)
        else if color[v] == 1:
            print CYCLIC and stop

print ACYCLIC
```

5.6 Correctness proof

Lemma 1. If the algorithm sees an edge to a color-1 vertex, the graph has a directed cycle.

Proof. Color 1 means the vertex is currently on the active DFS stack. If the current vertex is u and the edge is $u \rightarrow v$, then the stack contains a directed path from v down to u . Adding edge $u \rightarrow v$ closes a directed cycle. $\square$

Lemma 2. If the graph has a directed cycle, DFS will eventually see an edge to a color-1 vertex.

Proof. Consider the first vertex of that cycle entered by DFS. Following cycle edges from it, DFS either enters the next unvisited cycle vertex or encounters a vertex already active. The cycle cannot be fully finished before its closing edge is scanned, because finishing the first cycle vertex requires finishing all descendants reached through its outgoing edges. Therefore the closing edge is eventually scanned while its target is still active. $\square$

Theorem. The algorithm prints CYCLIC exactly when the directed graph contains a directed cycle.

Proof. Lemma 1 proves every reported cycle is real. Lemma 2 proves every real directed cycle is detected. The outer loop starts DFS from every unvisited component. $\square$

5.7 Complexity

Each directed edge is scanned once.

```
Time:  $O(n + m)$   
Memory:  $O(n + m)$ 
```

5.8 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;
    vector<vector<int>> adj(n);
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        adj[u].push_back(v);
    }

    vector<char> color(n, 0);
    for (int start = 0; start < n; start++) {
        if (color[start] != 0) continue;
```

```

    color[start] = 1;
    vector<pair<int, int>> st = {{start, 0}};
    while (!st.empty()) {
        int u = st.back().first;
        int i = st.back().second;
        if (i == (int)adj[u].size()) {
            color[u] = 2;
            st.pop_back();
            continue;
        }
        int v = adj[u][i];
        st.back().second++;
        if (color[v] == 0) {
            color[v] = 1;
            st.push_back({v, 0});
        } else if (color[v] == 1) {
            cout << "CYCLIC\n";
            return 0;
        }
    }
}

cout << "ACYCLIC\n";
return 0;
}

```

5.9 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    m = next(it)
    adj = [[] for _ in range(n)]
    for _ in range(m):
        u = next(it) - 1
        v = next(it) - 1
        adj[u].append(v)

    color = [0] * n
    for start in range(n):
        if color[start] != 0:
            continue
        color[start] = 1

```

```

stack = [(start, 0)]
while stack:
    u, i = stack[-1]
    if i == len(adj[u]):
        color[u] = 2
        stack.pop()
        continue
    v = adj[u][i]
    stack[-1] = (u, i + 1)
    if color[v] == 0:
        color[v] = 1
        stack.append((v, 0))
    elif color[v] == 1:
        print("CYCLIC")
        return
    print("ACYCLIC")

if __name__ == "__main__":
    main()

```

5.10 Common wrong answers

- Treating any edge to a visited vertex as a directed cycle.
- Marking a vertex finished too early, before all outgoing edges are scanned.
- Forgetting disconnected components.
- Reusing the undirected parent-skip rule for directed graphs.

5.11 Tests that matter

Use:

- directed acyclic graph with converging paths;
- simple directed cycle;
- cycle in a component not reachable from vertex 1;
- graph with no edges;
- repeated edge in an acyclic graph.

6 D. Component Audit

6.1 Restatement

The input is an undirected simple graph that can be disconnected. Classify every connected component as either a tree component (it has no cycle) or a cyclic component (it has at least one cycle), then print the two counts.

6.2 Constraints and traps

Both the number of vertices and roads can be 200000, so repeatedly scanning the graph for every component, or doing a separate cycle search after every traversal, risks unnecessary work. Isolated vertices count: a one-vertex, zero-edge component is a tree. Every undirected road appears in two adjacency lists, so summing degrees counts each road twice.

The graph is simple: there are no self-loops or repeated roads. That makes the standard connected-component characterization especially clean:

```
connected component is a tree <=> number of edges = number of vertices - 1
```

6.3 Observation and walk-through

Run one iterative DFS from each unvisited vertex. During that traversal, increment **vertices** whenever a vertex is popped, and add the length of its adjacency list to **degree_sum**. At the end, the component has **edges = degree_sum / 2** roads.

For the sample, DFS from 1 sees vertices 1, 2, 3. Its degree sum is $1 + 2 + 1 = 4$, so it has two edges and is a tree because $2 = 3 - 1$. DFS from 4 sees a triangle: three vertices and degree sum 6, hence three edges. Since $3 \neq 3 - 1$, that component is cyclic. Finally, 7--8 is a second tree component.

6.4 Algorithm

```
build an undirected adjacency list
seen[v] = false for every vertex
tree_components = cyclic_components = 0

for every vertex start:
    if start is seen:
        continue
```

```

vertices = degree_sum = 0
mark start and push it onto a stack
while stack is not empty:
    u = pop stack
    vertices += 1
    degree_sum += number of neighbors of u
    for every neighbor v of u:
        if v is unseen:
            mark v
            push v

edges = degree_sum / 2
if edges == vertices - 1:
    tree_components += 1
else:
    cyclic_components += 1

print tree_components and cyclic_components

```

Mark a vertex when it is pushed, not when it is popped. Otherwise two neighbors can push the same vertex before either copy is processed.

6.5 Correctness proof

Lemma 1. One DFS started at an unvisited vertex visits exactly its connected component, and no other component.

Proof. Every visited vertex is reached through a chain of graph edges from the start, so it belongs to the start's component. Conversely, DFS follows every edge incident to every vertex it visits. Any vertex connected to the start has a path from it; proceeding along that path shows DFS eventually marks every vertex on the path, including the endpoint. $\square$

Lemma 2. After one DFS, `vertices` and `degree_sum / 2` are respectively the number of vertices and edges in that connected component.

Proof. By Lemma 1, each component vertex is popped exactly once and adds one to `vertices`. Every component road appears once in the adjacency list of each endpoint, so it contributes exactly two to `degree_sum`. $\square$

Lemma 3. A connected component is a tree exactly when its edge count is one less than its vertex count.

Proof. A tree with V vertices has $V-1$ edges: start with one vertex and each newly attached vertex contributes one edge. Conversely, a connected graph with $V-1$ edges cannot contain a cycle. Removing an edge from a cycle keeps the graph connected, which would leave a connected graph with only $V-2$ edges; a connected graph needs at least $V-1$ edges to reach all V vertices. Thus it is acyclic and therefore a tree. $\square$

Theorem. The algorithm prints the correct number of tree and cyclic components.

Proof. The outer loop starts exactly one DFS for each connected component. By Lemma 2 it computes that component's exact vertex and edge counts, and by Lemma 3 it classifies the

component correctly. Adding one to the matching counter for every component gives exactly both requested totals. $\square$

6.6 Complexity

Every vertex is pushed once and every undirected road is scanned from each of its two endpoints.

```
Time:     $O(n + m)$ 
Memory:  $O(n + m)$ 
```

6.7 C++ implementation notes

Store `degree_sum` in `long long`. It fits in `int` under these bounds, but the type makes the relationship to an edge count explicit and remains safe if the exercise limits later grow. A `vector<char>` is enough for the `seen` state.

6.8 Python implementation notes

Use a normal list as a stack with `append` and `pop`; both are amortized constant time. Read all integers from `sys.stdin.buffer` to avoid repeated input overhead on the scale case.

6.9 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    cin >> n >> m;
    vector<vector<int>> graph(n);
    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    vector<char> seen(n, false);
    int tree_components = 0, cyclic_components = 0;
```

```

for (int start = 0; start < n; ++start) {
    if (seen[start]) continue;

    long long vertices = 0, degree_sum = 0;
    vector<int> stack = {start};
    seen[start] = true;
    while (!stack.empty()) {
        int u = stack.back();
        stack.pop_back();
        ++vertices;
        degree_sum += graph[u].size();
        for (int v : graph[u]) {
            if (!seen[v]) {
                seen[v] = true;
                stack.push_back(v);
            }
        }
    }

    if (degree_sum / 2 == vertices - 1) ++tree_components;
    else ++cyclic_components;
}

cout << tree_components << ' ' << cyclic_components << '\n';
}

```

6.10 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[0], data[1]
    graph = [[] for _ in range(n)]
    index = 2
    for _ in range(m):
        u = data[index] - 1
        v = data[index + 1] - 1
        index += 2
        graph[u].append(v)
        graph[v].append(u)

    seen = [False] * n
    tree_components = cyclic_components = 0
    for start in range(n):
        if seen[start]:
            continue

```

```

vertices = degree_sum = 0
stack = [start]
seen[start] = True
while stack:
    u = stack.pop()
    vertices += 1
    degree_sum += len(graph[u])
    for v in graph[u]:
        if not seen[v]:
            seen[v] = True
            stack.append(v)

if degree_sum // 2 == vertices - 1:
    tree_components += 1
else:
    cyclic_components += 1

print(tree_components, cyclic_components)

if __name__ == "__main__":
    main()

```

6.11 Common wrong answers

- Starting DFS only from vertex 1, which ignores other components.
- Calling every component with at least one edge cyclic; a path has edges but is still a tree.
- Comparing `degree_sum` directly with `vertices - 1` instead of dividing by two.
- Marking vertices only on pop, allowing duplicate stack entries and corrupting the collected statistics.

6.12 Tests that matter

- no roads: every isolated vertex is a tree component;
- one triangle plus a path: checks both categories in one graph;
- a cycle with a long tail: it remains one cyclic component;
- nested-looking traversal order with many cross edges: edge counting must not depend on DFS parent choices;
- a 200000-vertex cycle: rejects recursive implementations that exhaust the call stack and quadratic repeated traversals.

7 Stress testing

Good stress tests for this section compare independent ideas:

Problem A:

```
generate a random tree
compute subtree sizes with the iterative method
compare with a slow recursive method on small n
```

Problem B:

```
for each connected component, count vertices and edges
in a simple undirected graph, a connected component is cyclic iff edges >= vertices
```

Problem C:

```
compare DFS colors against Floyd-Warshall reachability on tiny directed graphs:
there is a cycle iff some edge  $u \rightarrow v$  has  $v$  reachable back to  $u$ 
```

Problem D:

```
brute-force every connected component with a tiny BFS
count its vertices and unique undirected edges
classify it using edges == vertices - 1
```

The local generators use direct reference implementations and include both fixed edge cases and randomized cases for C++ and Python.

8 What to carry forward

DFS is the tool to reach for when:

```
the graph is unweighted but shortest distance is not the point  
the problem asks about descendants or subtrees  
you need to know when a branch is finished  
you need to detect cycles
```

The next section builds on DFS finishing order and introduces topological sorting and dynamic programming on DAGs.